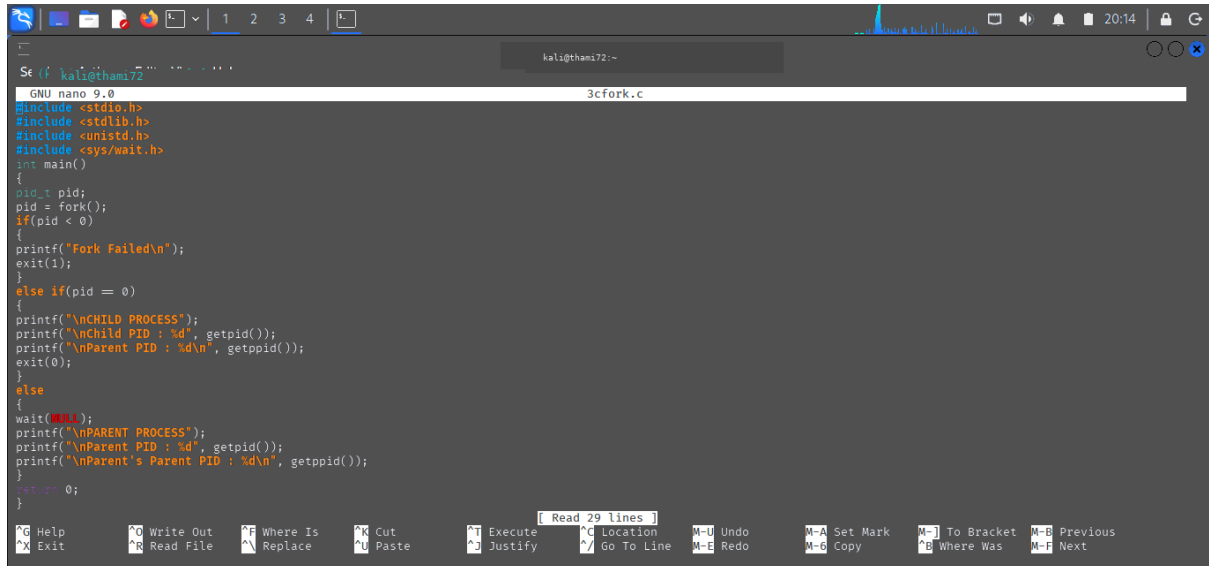


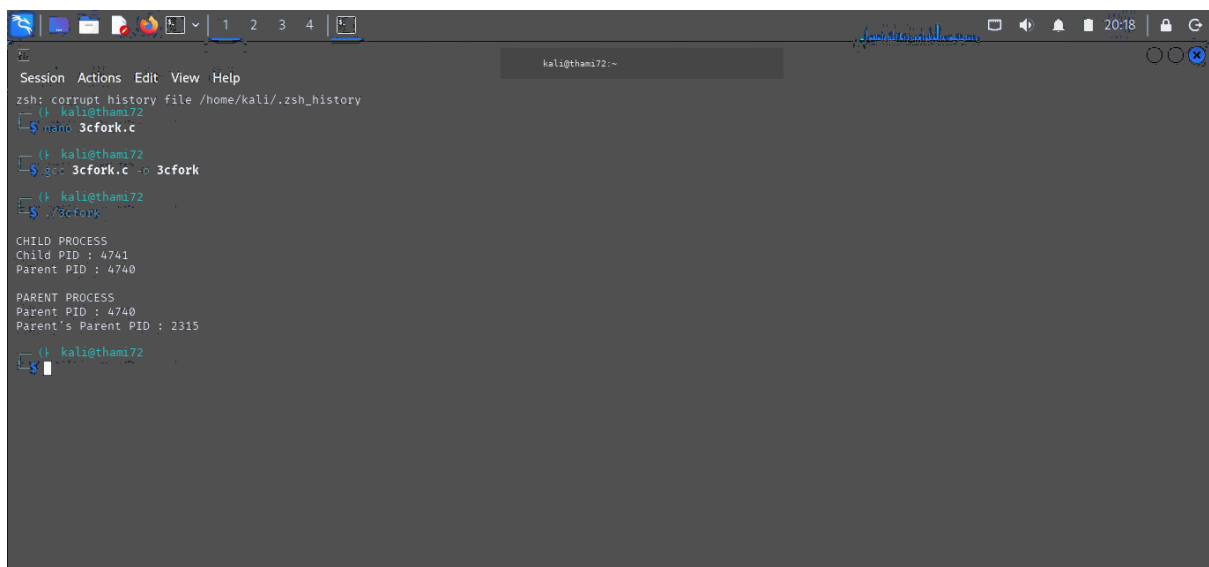
C PROGRAMMING : fork()



```
GNU nano 9.0 3cfork.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

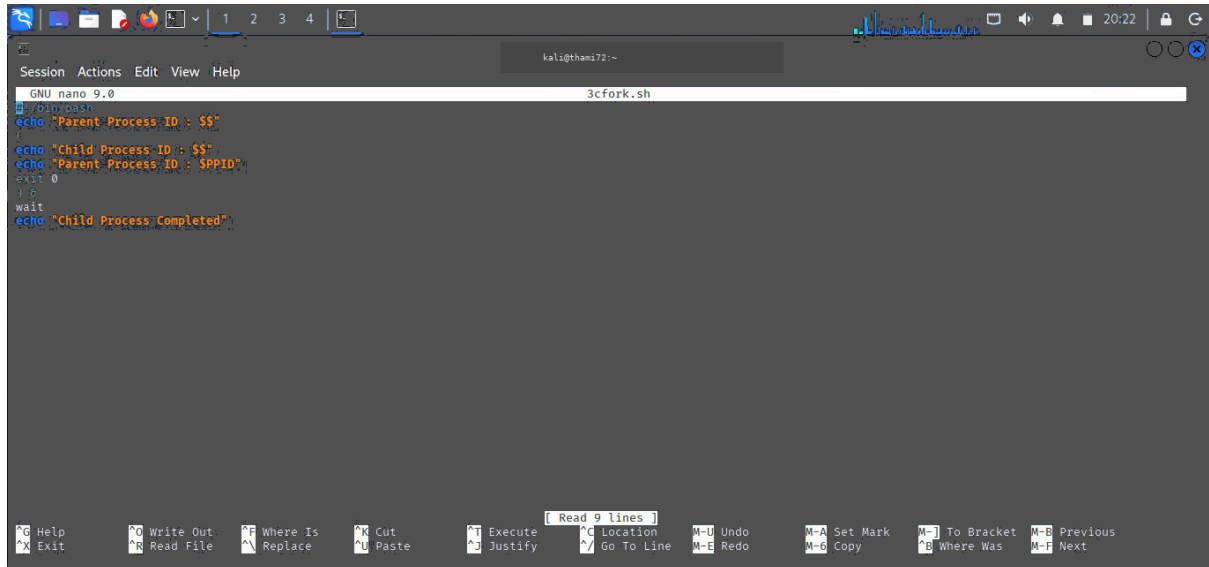
int main()
{
    pid_t pid;
    pid = fork();
    if(pid < 0)
    {
        printf("Fork Failed\n");
        exit(1);
    }
    else if(pid == 0)
    {
        printf("\nCHILD PROCESS");
        printf("\nChild PID : %d", getpid());
        printf("\nParent PID : %d\n", getppid());
        exit(0);
    }
    else
    {
        wait(NULL);
        printf("\nPARENT PROCESS");
        printf("\nParent PID : %d", getpid());
        printf("\nParent's Parent PID : %d\n", getppid());
    }
    return 0;
}
```

OUTPUT :



```
Session Actions Edit View Help
zsh: corrupt history file /home/kali/.zsh_history
kali@thami72:~$ nano 3cfork.c
kali@thami72:~$ gcc 3cfork.c -o 3cfork
kali@thami72:~$ ./3cfork
CHILD PROCESS
Child PID : 4741
Parent PID : 4740
PARENT PROCESS
Parent PID : 4740
Parent's Parent PID : 2315
kali@thami72:~$
```

SHELL PROGRAMMING :fork()

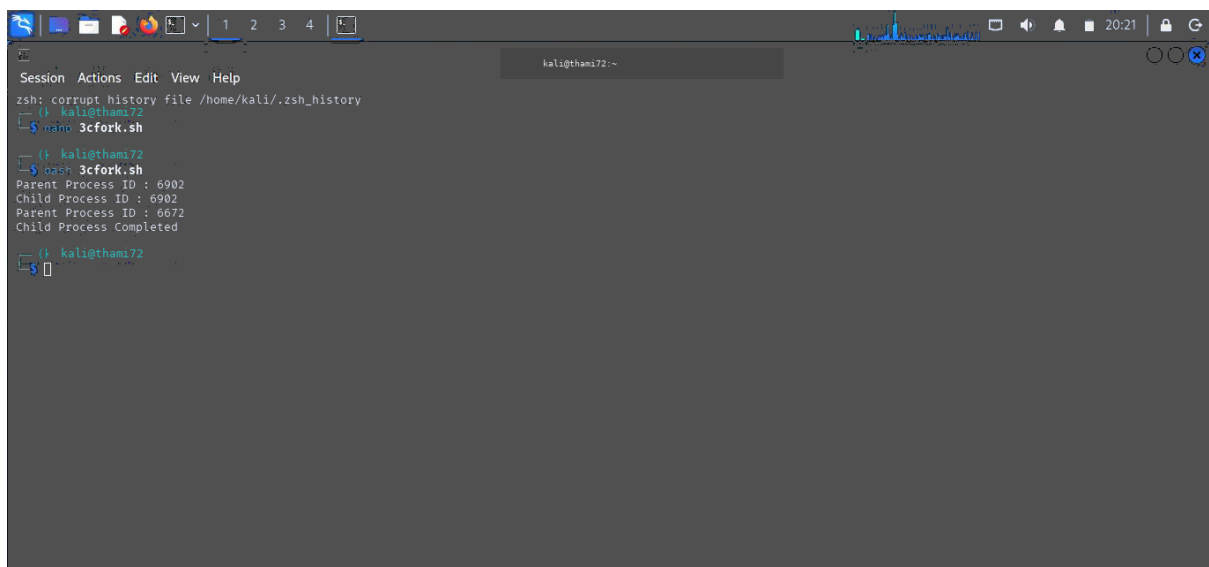


A screenshot of a terminal window with the nano 9.0 editor open. The editor is editing a file named 3cfork.sh. The script content is as follows:

```
#!/bin/bash
echo "Parent Process ID : $$"
{
echo "Child Process ID : $$"
echo "Parent Process ID : $PPID"
exit 0
} &
wait
echo "Child Process Completed"
```

The terminal window title is kali@thami72:~. The nano editor interface includes a menu bar (Session, Actions, Edit, View, Help) and a status bar at the bottom with various keyboard shortcuts.

OUTPUT :

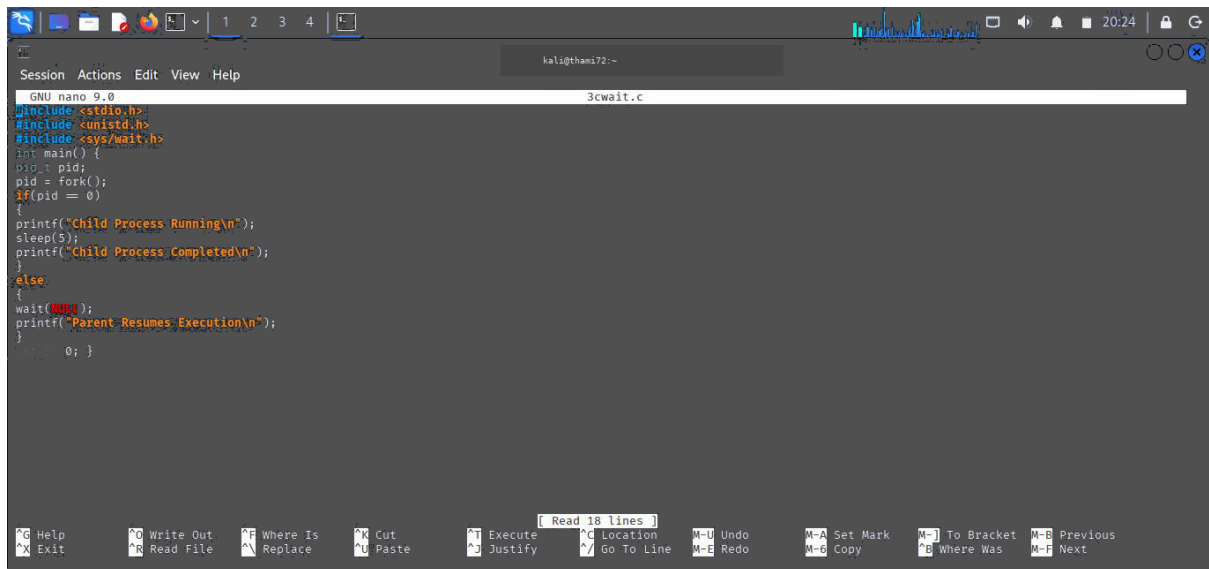


A screenshot of a terminal window showing the execution of the 3cfork.sh script. The terminal output is as follows:

```
zsh: corrupt history file /home/kali/.zsh_history
kali@thami72 ~$ nano 3cfork.sh
kali@thami72 ~$ bash 3cfork.sh
Parent Process ID : 6902
Child Process ID : 6902
Parent Process ID : 6672
Child Process Completed
kali@thami72 ~$
```

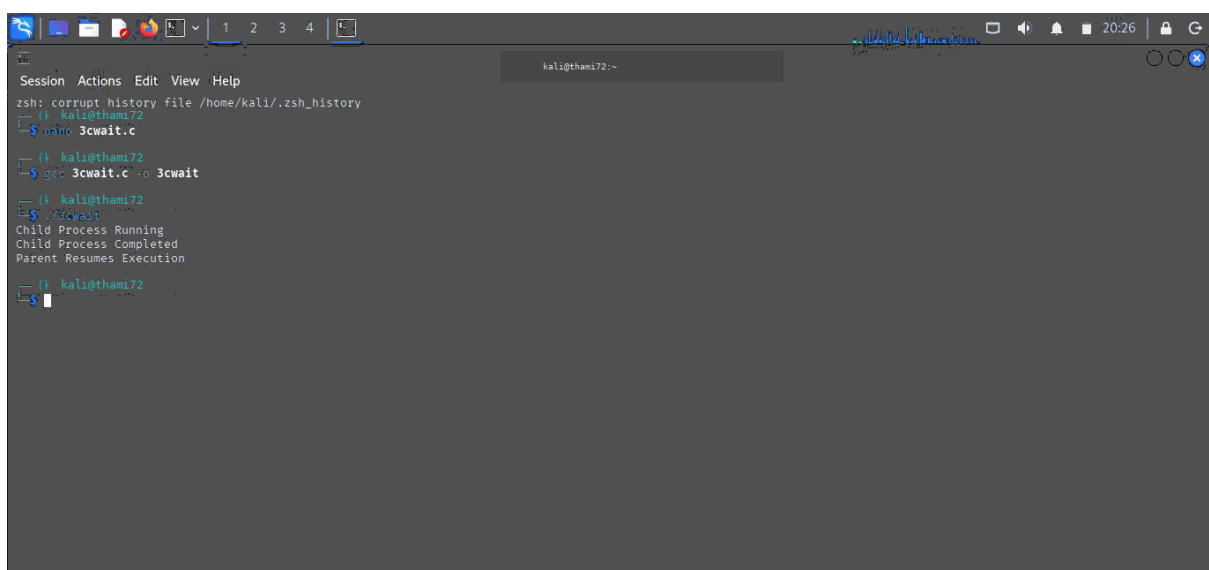
The terminal window title is kali@thami72:~. The output shows the parent process ID (6902), the child process ID (6902), and the parent process ID (6672) after the child process has completed.

C PROGRAMMING :wait()



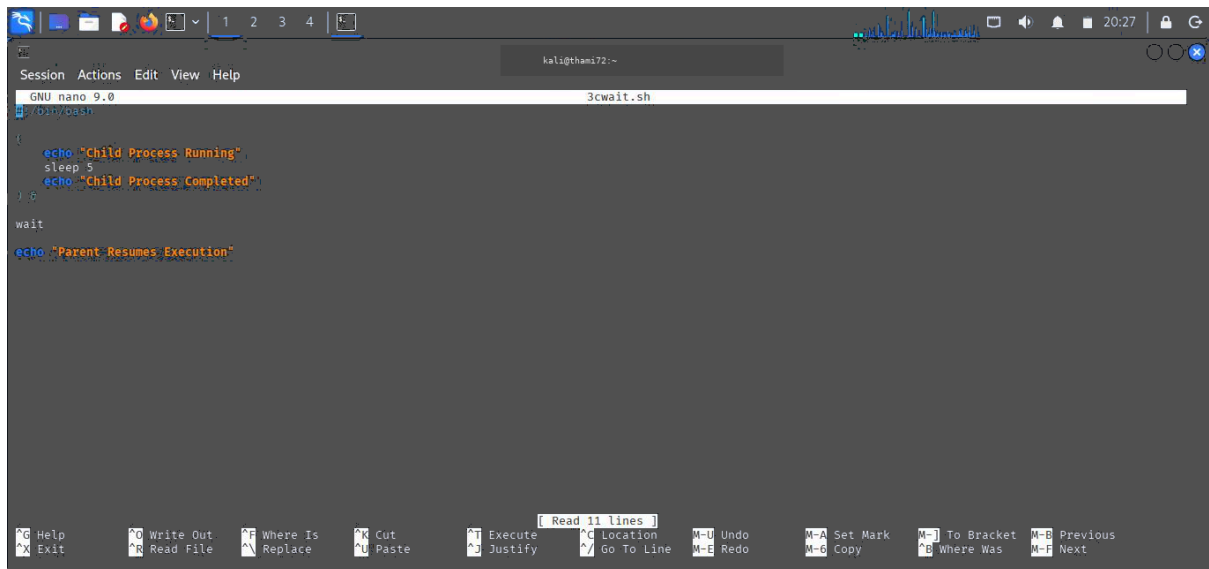
```
GNU nano 9.0 3cwait.c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main() {
    pid_t pid;
    pid = fork();
    if (pid == 0) {
        printf("Child Process Running\n");
        sleep(5);
        printf("Child Process Completed\n");
    } else {
        wait(NULL);
        printf("Parent Resumes Execution\n");
    }
    return 0; }
```

OUTPUT :



```
kali@thami72:~$ zsh: corrupt history file /home/kali/.zsh_history
kali@thami72:~$ nano 3cwait.c
kali@thami72:~$ gcc 3cwait.c -o 3cwait
kali@thami72:~$ ./3cwait
Child Process Running
Child Process Completed
Parent Resumes Execution
kali@thami72:~$
```

SHELL PROGRAMMING :wait()



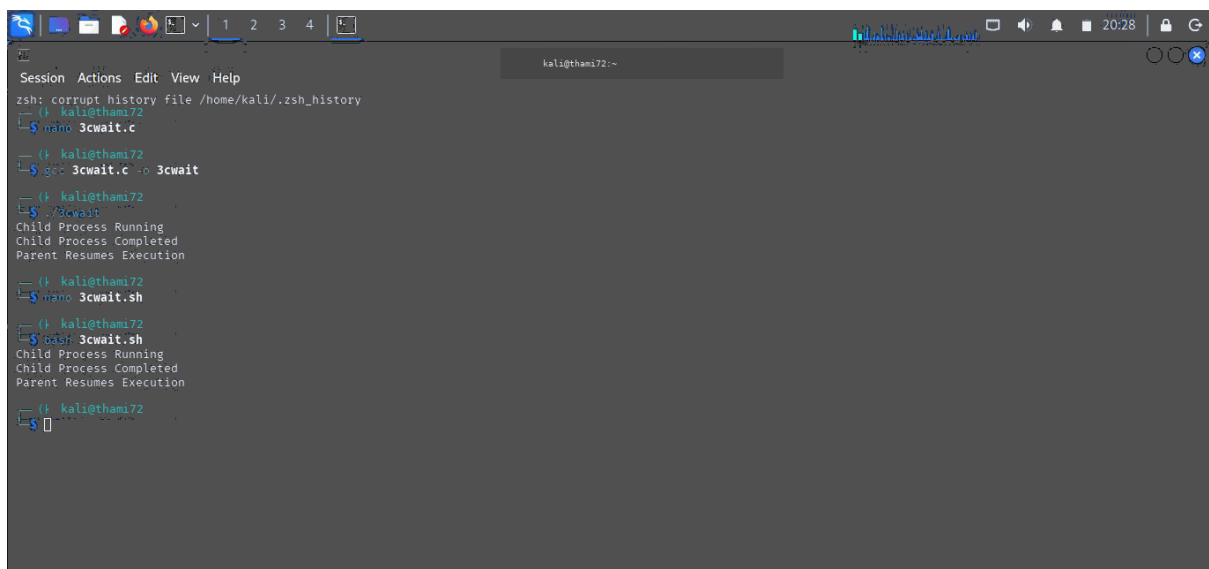
```
GNU nano 9.0 3cwait.sh
#!/bin/bash

echo "Child Process Running"
sleep 5
echo "Child Process Completed"

wait

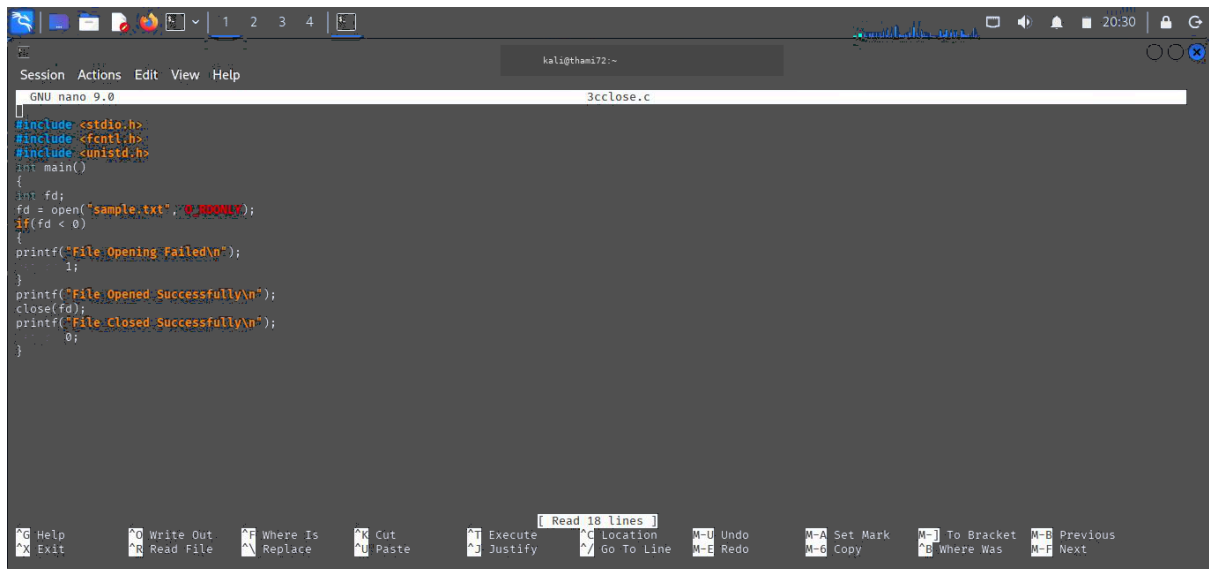
echo "Parent Resumes Execution"
```

OUTPUT :



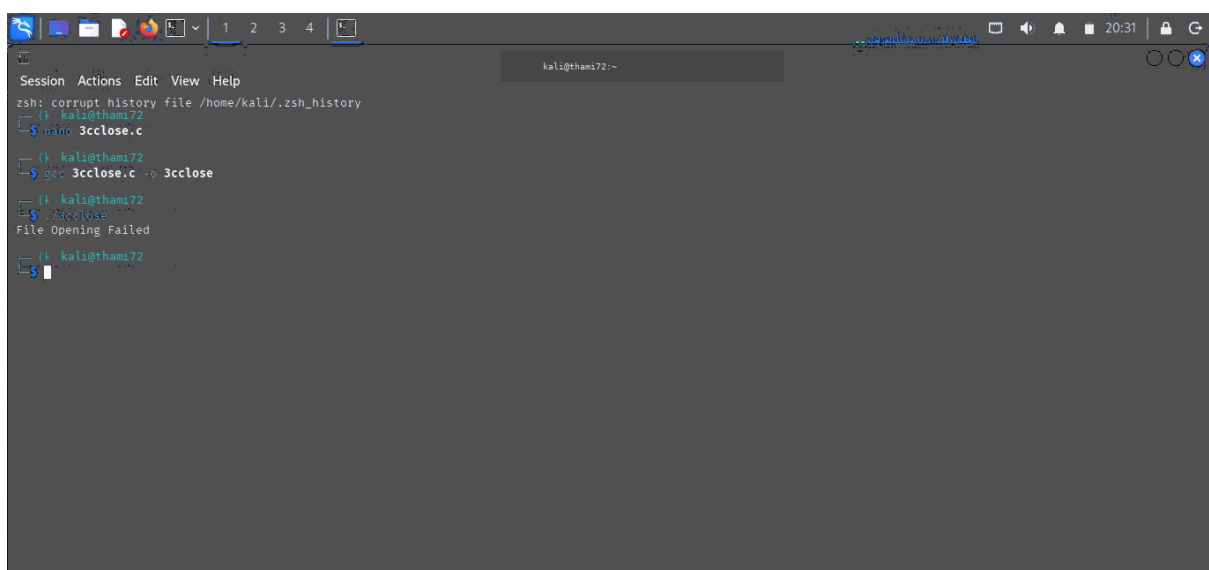
```
kali@thami72:~$ gcc 3cwait.c -o 3cwait
kali@thami72:~$ ./3cwait
Child Process Running
Child Process Completed
Parent Resumes Execution
kali@thami72:~$ nano 3cwait.sh
kali@thami72:~$ bash 3cwait.sh
Child Process Running
Child Process Completed
Parent Resumes Execution
kali@thami72:~$
```

C PROGRAMMING :close()



```
GNU nano 9.0 3cclose.c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
int main()
{
    int fd;
    fd = open("sample.txt", O_APPEND);
    if(fd < 0)
    {
        printf("File Opening Failed\n");
        return 1;
    }
    printf("File Opened Successfully\n");
    close(fd);
    printf("File Closed Successfully\n");
    return 0;
}
```

OUTPUT :



```
zsh: corrupt history file /home/kali/.zsh_history
kali@thami72:~$ nano 3cclose.c
kali@thami72:~$ gcc 3cclose.c -o 3cclose
kali@thami72:~$ ./3cclose
File Opening Failed
kali@thami72:~$
```

SHELL PROGRAMMING :close()

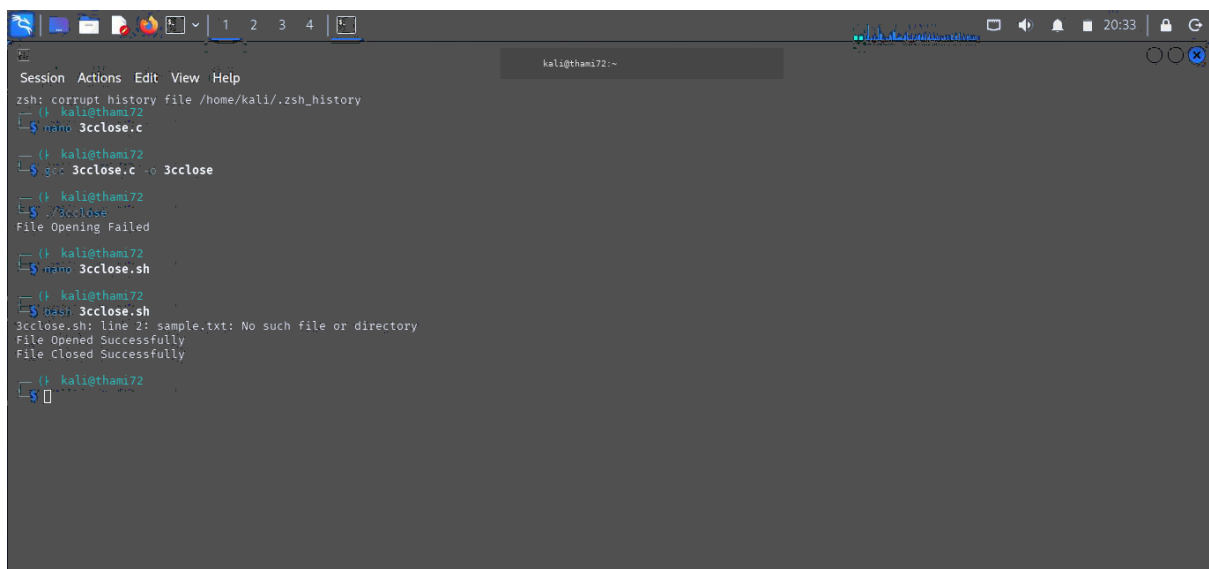


A screenshot of a terminal window with the nano 9.0 editor open. The editor is editing a file named 3cclose.sh. The content of the file is as follows:

```
#!/bin/bash
exec 3< sample.txt
echo "File Opened Successfully"
exec 3<&=
echo "File Closed Successfully"
```

The terminal window title is 'kali@thami72:~'. The nano editor's status bar at the bottom shows various keyboard shortcuts like 'G Help', 'X Exit', 'O Write Out', 'R Read File', 'F Where Is', 'N Replace', 'C Cut', 'P Paste', 'T Execute', 'J Justify', 'L Read 5 lines', 'G Location', 'C Go To Line', 'U Undo', 'R Redo', 'A Set Mark', 'B Copy', 'I To Bracket', 'B Where Was', 'P Previous', and 'F Next'.

OUTPUT :



A screenshot of a terminal window showing the execution of the 3cclose.sh script. The terminal output is as follows:

```
zsh: corrupt history file /home/kali/.zsh_history
kali@thami72:~$ nano 3cclose.c
kali@thami72:~$ gcc 3cclose.c -o 3cclose
kali@thami72:~$ ./3cclose
File Opening Failed
kali@thami72:~$ nano 3cclose.sh
kali@thami72:~$ bash 3cclose.sh
3cclose.sh: line 2: sample.txt: No such file or directory
File Opened Successfully
File Closed Successfully
kali@thami72:~$
```

The terminal window title is 'kali@thami72:~'. The output shows the script being compiled with gcc, executed with ./3cclose, and then with bash 3cclose.sh. The script attempts to open sample.txt for reading, but it fails because the file does not exist. The script then prints 'File Opened Successfully' and 'File Closed Successfully'.